

sudoku practice ungraded assignment consists of two tasks:

In **Task 1** (solver implementation), you will implement from scratch an AI agent to solve sudoku puzzles using any algorithm of your choice. You need to only consider 9×9 sudoku grids where each sudoku has either a single solution or no solution.

In Task 2 (report), you will write a 1-page report to explain and discuss your solution and practice referencing using harvard-Bath style.

Task description and deliverables

Task 1 (solver implementation) where most of your time should be spent. You will be given 1) a starter notebook sudoku.ipynb detailing the required implementation template and 2) a set of sudoku puzzles for testing. You will write and validate your solver implementation in the notebook provided following the instructions in it, crucially including those on ensuring the compatibility with the automated testing script.

Important note: Your tutor will test all submissions with numpy 1.26.4 and python 3.10 as default. If you do not use the same configuration, the performance of your solver can be negatively affected during testing. To avoid that, please ensure you use the stated numpy and python versions to write your implementation (or do get in touch with us well in advance of submission in case of any insurmountable problems with this requirement).

The choice of algorithm to solve sudoku puzzles is up to you, provided you can implement it yourself from scratch. We expect you will use search algorithms taught in the unit, but you are welcome to implement your own ideas or do independent research for suitable approaches. There is often a trade-off between performance and complexity of implementation.

You could for example implement an algorithm that treats sudoku as a constraint satisfaction problem (CSP) as it is taught in the unit. You may use any relevant code you have previously worked with in ungraded practicals of this unit as a guide for your implementation. A good implementation of backtracking depth-first search for CSPs with constraint propagation would make a strong submission. There are also both easier and more complex options for your solver approach. Implementation of only one algorithm is required and will be assessed as your submission in the automated testing. If you experiment with more than one algorithm, please ensure that you submit only your best performing one at the end.

You are encouraged to implement your chosen algorithm efficiently, using standard python. However, this assignment is not about multi-threading, use of accelerated python or the GPU. Such methods to gain a speed advantage will not be accepted, and you may get flagged and asked to redo if such an unauthorised enhancement cannot be easily disabled in your code before testing. Your solution must be designed to run in the default single-core jupyter notebook environment.

The deliverable for Task 1 (solver implementation) is the sudoku.ipynb jupyter notebook with your solver implementation using python 3.10 and numpy 1.26.4, validated using pre-submission tests detailed in the notebook.

Task 2 (report). For this task, you need to write a 1-page report to present and discuss your implemented solver solution. The page limit is strict, but excludes references. The report must consist of the following sections:

- Statement of approach: Using proper terminology, provide a statement of the approach you have taken to solve the task.
- Core algorithm description: A description of the core algorithm you have implemented in technically sound terminology and, where appropriate, formalised with supporting mathematical notation and/or diagrams.
- Optimisations: A statement of all employed optimisations to the core algorithm and/or its basic implementation. Highlight the contribution of each optimisation to reducing the overall search complexity (your quantitative results must reflect the claims made).
- Reflections and suggestions for further work: Provide an assessment of your solver's performance with reference to suitable quantitative and qualitative metrics. Both strengths and limitations should be part of the assessment. Propose any future work that could address the current limitations.

The deliverable for Task 2 (report) is the text file report.pdf

Tutor Assessment Criteria

Task 1 (solver implementation) will be assessed using automated testing based on the key criteria of correctness and speed of sudoku puzzle solving. The automarker will use a set of test sudoku puzzles of various difficulty, some of which are in your given data (provided set) but most of which are not (hidden set).

It is recommended that you ensure correctness of your solver for all in-scope scenarios before you start optimising it for the speed criterion.

For the speed criterion, the simple guideline is the faster, the better. The automarker operates with an elaborate system of timeouts (both for each puzzle and each difficulty level) and failure limits (both time and correctness-wise), as it runs your solver on the test set of puzzles. How well you do is determined by how far your solver gets through this obstacle course. The script begins testing from the very easy category up to the very hard category.

The absolute upper bound on solving time is 30 seconds i.e., the automarker will never give any credit for a puzzle taking longer than that. If you consistently get per-puzzle latencies close to the upper limit, you should re-consider your approach. On the other hand, you know you are on the right track to a good implementation if you are getting an average latency of under one second per puzzle. There will naturally be differences in average latencies for different difficulty levels: a fairly standard solver implementation may get 0.001 seconds per puzzle for the very easy sudoku category, but struggle to finish under the upper bound of 30 seconds for the hard sudokus.

Finally, please be mindful that the measured latencies will vary with hardware. The automarker will be run on the same test machine to ensure consistent relative performance of all submissions and grade fairness. However, the absolute speeds we measure on the test machine may not necessarily be the same as observed on your machine: for example, if

yours is a particularly advanced hardware one, you may get a favourably misrepresented speed performance relative to our test machine. The safeguard against falling into the trap is 1) to have an objective theory-based expectation of the performance of your approach and 2) not to stop optimising it too prematurely based just on the absolute speeds observed.

Task 2 (report) will be assessed as follows:

Statement of approach

- The objective difficulty of coherently stated approaches will be assessed and credited by the marker;
- Marks will be deducted for incorrect use of terminology in the statement of approach;
- Marks will be deducted if there is an inconsistency between what is claimed to be the approach in the report and the observed performance of the solver.

Core algorithm description

- How well understood and explained is the algorithm that has been implemented?
- Is the explanation suitably conceptually generalised or, on the contrary, too low-level, detailing the algorithm in terms of the functions of own implementation?
- Is the core algorithm explained separately from the optimisations?

How well have the optimisations been explained?

- How sophisticated and numerous have the optimisations been?
- How successful have the optimisations been? Highlight the contribution of each optimisation to reducing the overall search complexity, both qualitatively and with reference to your empirical solver performance results. If you attempt formal theoretical complexity reduction analysis using complexity measures for bonus credit, make sure the statements are fully mathematically formalised and derived in the report: quantitative statements on complexity metrics of optimised solutions quoted without a full derivation will not be credited.

Reflections and suggestions for further work

- How well has the success of your approach been evaluated?
- Have the strengths and limitations been explicitly identified?
- Has sensible and interesting future work been proposed to address specific limitations?
- Have the future work methods proposed been conceptually defined?